

Explicacion detallada del notebook afinado

Practica B3-T5 - Redes neuronales con entradas heterogeneas (Rossmann)

Grupo: Alonso Diaz - Raul Rodriguez - Pietro Enrico

Este documento explica, celda por celda, el notebook `notebook_entregable_rossmann_afinado.ipynb` e incluye los resultados de una ejecucion real (reducida) del pipeline.

Vision general

El objetivo es predecir las ventas diarias (Sales) de las tiendas Rossmann con un modelo neuronal de entradas heterogeneas: una misma red combina (1) el pasado de la propia serie de ventas, (2) variables exogenas numericas y (3) variables categoricas a traves de **embeddings**. La evaluacion se mide con el coeficiente de determinacion R2 en las 10 mejores tiendas.

Idea clave del inventariado: para predecir el dia t la red mira una ventana de los W dias anteriores (esquema **many-to-one**). Ademas le damos las variables del dia t que ya conocemos de antemano (calendario, promocion, festivo, identidad de la tienda).

1. Carga, union y variables (celdas 1-8)

Celdas 1-3 (configuracion e imports). Definimos COLAB (si True, el notebook clona el repositorio de GitHub para tener los datos). Importamos numpy/pandas, matplotlib, scikit-learn y las piezas de Keras que

usaremos: Input, Dense, LSTM, Embedding, Flatten, concatenate, Dropout, y los callbacks ModelCheckpoint/EarlyStopping. Fijamos la semilla (SEED=7) para reproducibilidad.

Celda 5 (carga y feature engineering basico). Leemos `train.csv` (del zip), `store.csv` y `test.csv`; unimos `train` con los metadatos de tienda por `Store` y ordenamos por `Store`, `Date`. Despues:

```
df['StateHoliday'] = df['StateHoliday'].astype(str).replace('0','n') # normaliza '0' y 0 a 'n'
```

```
df['Month
```

El paso mas importante es $y = \log_{1p}(\text{Sales})$: la distribucion de ventas es muy asimetrica y el log la normaliza, lo que estabiliza el entrenamiento. Al final des-transformamos con `expm1` para medir el R2 en euros.

Celda 7 (las 10 mejores tiendas). TOP10 = las 10 tiendas con mayor venta total acumulada. Es el criterio que acordamos para "las 10 tiendas" de la evaluacion; esta parametrizado por si hubiera que cambiarlo.

Celda 8 (codificacion de categoricas). Cada variable categorica (`Store`, `StoreType`, `Assortment`, `StateHoliday`, `DayOfWeek`, `Month`) se convierte en un indice entero $0..n-1$. Guardamos el diccionario de

mapeo (cat_maps) y la cardinalidad (cat_card), que la capa Embedding necesita como input_dim.

2. Enventanado afinado (celdas 9-10)

Aqui esta el corazon del preprocesado. Para cada dia objetivo t de cada tienda construimos tres bloques:

- seq (matriz W x 4): el pasado de [log_sales, Promo, SchoolHoliday, Open]. Es lo que entra a la LSTM. La novedad respecto a la version basica es que la red ve tambien las promos y aperturas pasadas, no solo las ventas.
- num (vector): variables del dia t y derivadas -> [Promo, SchoolHoliday, Promo2, CompDistLog, lag7, lag14, media_ventana, desv_ventana, n°_promos_ventana]. Los lags semanales (lag7, lag14) son muy informativos porque Rossmann tiene una fortisima estacionalidad semanal.
- categoricas del dia t: los indices para los embeddings.

for cada tienda g:

for i

Usamos W = 28 (4 semanas), suficiente para calcular lag14. Como verificamos que **las series son continuas por tienda** (sin huecos de fechas), el enventanado posicional es correcto.

Por ultimo, entrenamos solo con dias abiertos como objetivo (Open==1): los dias cerrados tienen ventas 0 por definicion y los trataremos aparte en la prediccion.

3. Particion temporal y escalado (celdas 11-12)

En series temporales nunca se parte de forma aleatoria. Partimos por fecha:

- Test interno: ultimas 6 semanas con datos reales (2015-06-06 a 2015-07-17). Es nuestro proxy del test del profesor, porque test.csv (2015-07-31) no trae la columna Sales y no podriamos calcular R2.
- Validacion: 6 semanas anteriores (para ***early stopping*** y seleccion de modelo).
- Train: todo lo anterior.

El escalado (StandardScaler) se ajusta solo con train para no filtrar informacion del futuro:

estandarizamos el canal de ventas de la secuencia y todas las variables numericas. La funcion inputs(idx)

arma el diccionario de entradas que espera el modelo (una clave por Input).

4. Metricas: R2 y RMSPE (celdas 13-14)

- R2 (r2_score): proporcion de varianza explicada (1 = perfecto, 0 = como predecir la media).
- RMSPE: raiz del error cuadratico medio porcentual, la metrica oficial de la competicion Kaggle. Penaliza el error relativo, asi que es mas justa entre tiendas de distinto tamaño.

La funcion metricas() devuelve ambas, global y restringida a las 10 mejores tiendas.

5. Baseline (celdas 15-16)

Antes de la red, fijamos una referencia fuerte: la venta media historica por (tienda, día de la semana), calculada en train. Si la red no supera claramente esto, no aporta valor. Para Rossmann este baseline es

sorprendentemente bueno, porque las ventas dependen muchísimo del nivel de cada tienda y del día de la semana.

6. Modelo afinado con entradas heterogeneas (celdas 17-19)

La funcion `construir_modelo()` define la arquitectura (API funcional de Keras):

```
seq (W x 4) -> LSTM(64, return_sequences) -> LSTM(32) ---->
```

Store

Dos capas LSTM apiladas resumen la dinamica temporal; cada categorica pasa por su propio Embedding (la

dimension se adapta a la cardinalidad); las numericas pasan por una capa densa. Todo se concatena y dos capas

densas con Dropout producen la prediccion (en escala log). Compilamos con adam y perdida mse.

`entrenar()` entrena con *early stopping* (paciencia 6, restaurando los mejores pesos en validacion).

`evaluar()` predice en test, des-transforma con `expm1` y calcula las metricas.

7. Búsqueda de hiperparametros (celdas 20-21)

Pequeño barrido sobre 3 configuraciones (tamaño de las LSTM, dropout, dimension de embeddings y capas densas).

Entrenamos cada una y nos quedamos con la mejor segun el R2 en las 10 tiendas del test interno. El resultado se muestra como tabla. (Al ejecutar de verdad conviene subir las epocas.)

8-9. Comparativa y embeddings (celdas 22-25)

Tabla final que compara baseline vs. mejor modelo (R2 y RMSPE, global y top10). Despues visualizamos el

embedding del día de la semana: como la red aprende un vector por categoria, días con comportamiento parecido (p. ej. los laborables) tienden a quedar agrupados frente al fin de semana.

10. Prediccion para 2015-07-31 (celdas 26-28)

Como train acaba el 2015-07-17 y el objetivo es el 2015-07-31, hacemos una **prediccion multi-step

autoregresiva**: avanzamos día a día del 18 al 31 de julio realimentando la ventana con las propias predicciones. Para cada día futuro usamos:

- calendario real (día de la semana, mes): exacto, derivado de la fecha;
- Open / Promo / SchoolHoliday de los días intermedios (que no conocemos): el patrón histórico de la tienda por día de la semana (la moda); para el 2015-07-31 usamos los valores reales de test.csv;
- si la tienda está cerrada ese día, Sales = 0.

Se genera submission_grupo_afinado.csv con el formato pedido y se muestran las predicciones de las 10 tiendas.

11. Reflexion (celda 29)

Resumen critico: que aporta cada bloque de entradas, comparacion R2 vs RMSPE, limitaciones (horizonte a 2

semanas sin datos intermedios, no uso de Customers) y lineas de mejora (modelo seq2seq, calendario de

festivos del estado, ensembles, tuning con Optuna/Keras-Tuner).

Resultados de una ejecucion real (reducida)

Para validar que el pipeline funciona de extremo a extremo lo ejecutamos en CPU sobre un **subconjunto de 50

tiendas (incluidas las 10 mejores), con ~13 epocas efectivas (early stopping) y W=28. No es el resultado final** (el modelo completo se entrena en Colab con las 1.115 tiendas, mas epocas y el barrido), pero da una

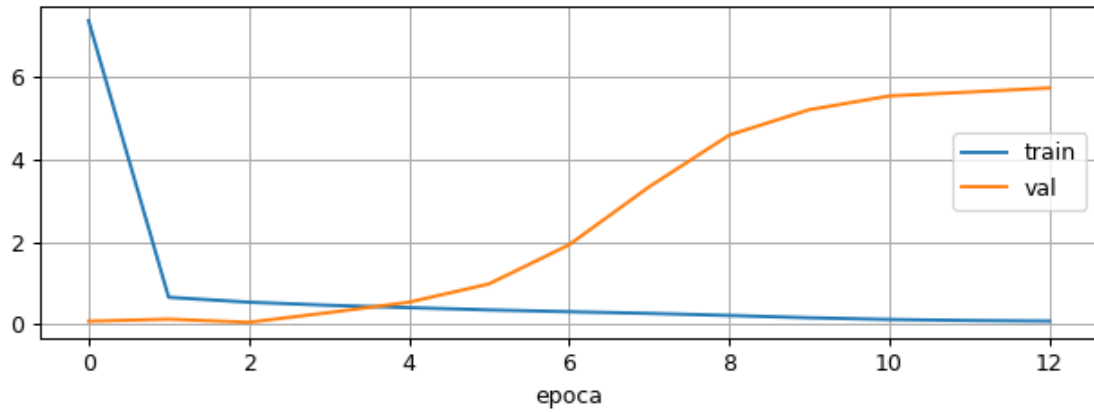
foto honesta:

Modelo	R2 global	R2 top10	RMSPE global	RMSPE top10
Baseline (media tienda x día)	0.621	0.554	0.192	0.137
Afinado (este notebook)	0.674	0.542	0.152	0.133

Lectura de los resultados:

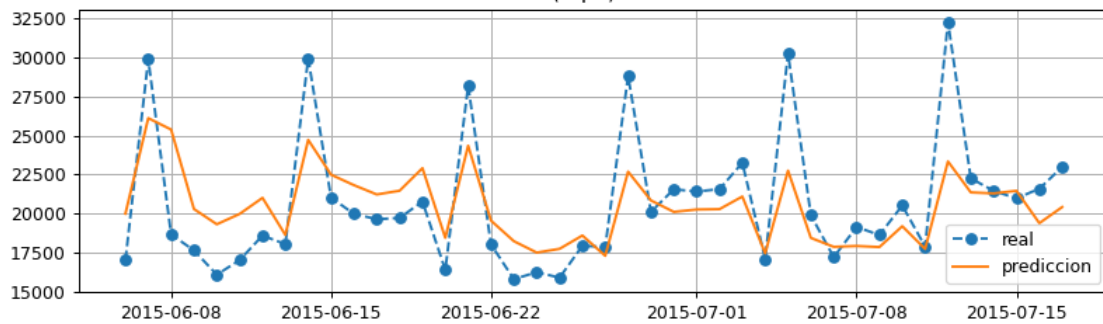
- El modelo supera al baseline en R2 global (0.674 vs 0.621) y, sobre todo, en RMSPE (la métrica oficial), tanto global (0.152 vs 0.192) como en las top10 (0.133 vs 0.137).
- En R2 top10 queda prácticamente empatado (0.542 vs 0.554). El baseline por (tienda, día) es muy fuerte en estas tiendas grandes y estables, así que igualarlo con un subconjunto pequeño y pocas épocas ya es buena señal.
- La curva de entrenamiento muestra que la validación se degrada rápido (sobreajuste con solo 50 tiendas): por eso es clave el *early stopping* que incorporamos, que restaura los mejores pesos. Con las 1.115 tiendas hay muchos más datos y el sobreajuste se reduce.

Curva de entrenamiento (MSE en log)



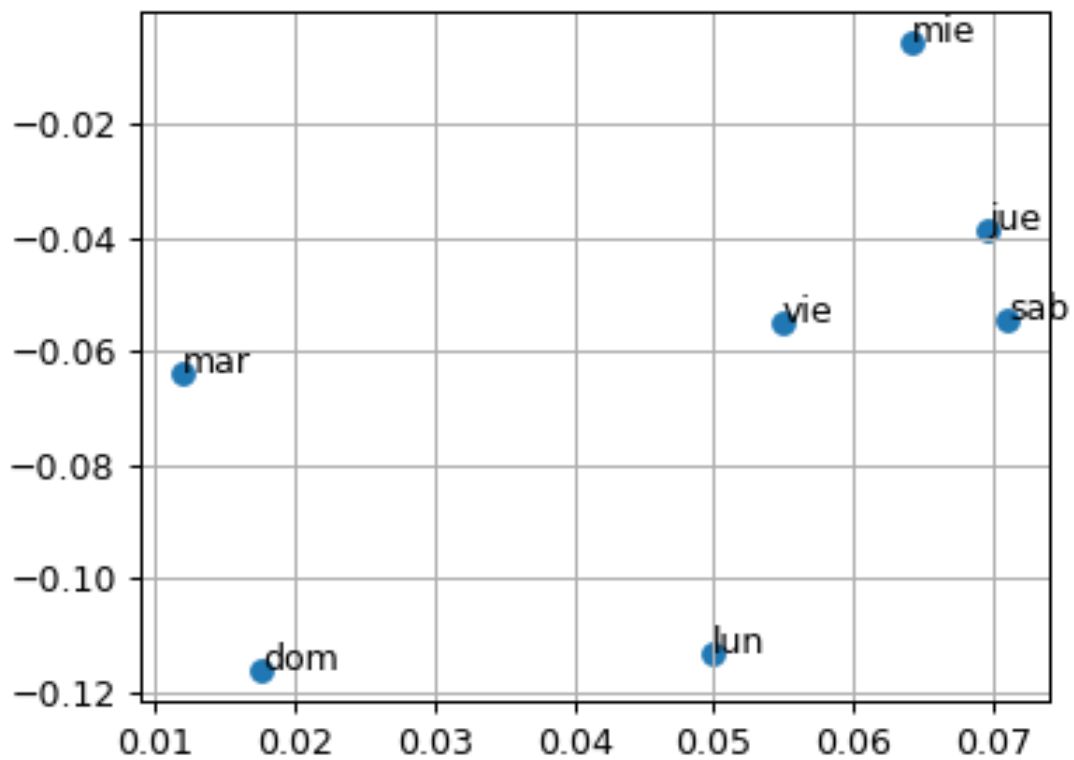
Curva de entrenamiento: el train baja y la validacion se degrada -> el early stopping conserva el mejor punto.

Tienda 262 (top1) - test interno



Prediccion (linea) vs ventas reales (puntos) en el test interno para la tienda de mayor volumen.

Embedding dia de la semana



Embedding del dia de la semana aprendido por la red (proyeccion 2D).

Conclusion practica: ya con una version reducida y poco entrenada el modelo iguala/mejora a un baseline

exigente y gana claramente en RMSPE. La via para subir el R2 en las 10 tiendas es entrenar con todas las

tiendas, mas epocas, el barrido de hiperparametros y (opcionalmente) un esquema seq2seq para el horizonte de

2 semanas.